

Develop a predictive text system using N-Gram and HMM (Hidden Markov Models) to predict the next word based on previous context.

Aim

To develop a predictive text system using **N-Gram** and **Hidden Markov Models (HMM)** to predict the next word based on previous context.

Dataset Brown Corpus

Step 1: Load the Dataset

```
import nltk
nltk.download('brown')

from nltk.corpus import brown
sentences = brown.sents()
```

Step 2: Text Preprocessing

Convert text to lowercase and remove special characters, punctuation, and extra spaces.

```
import re

clean_sentences = []
for sent in sentences:
    text = " ".join(sent).lower()
    text = re.sub(r'[^\w\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    clean_sentences.append(text)
```

Step 3: Tokenization

```
tokenized_sentences = []
for sent in clean_sentences:
    tokenized_sentences.append(sent.split())
```

Step 4: Create Vocabulary

```
vocab = set()
for sent in tokenized_sentences:
    for word in sent:
        vocab.add(word)
```

```
vocab_size = len(vocab)
```

N-GRAM MODEL Step 5: Define N-grams

(Bigram & Trigram)

```
from nltk.util import ngrams
```

```

bigrams = []
trigrams = []

for sent in tokenized_sentences:
    if len(sent) >= 2:
        bigrams.extend(list(ngrams(sent, 2)))
    if len(sent) >= 3:
        trigrams.extend(list(ngrams(sent, 3)))

```

Step 6: Generate Sentence using N-gram

```

generated_sentence = ['the']

for i in range(10):
    last_word = generated_sentence[-1]
    candidates = []

    for bg in bigrams:
        if bg[0] == last_word:
            candidates.append(bg[1])

    if len(candidates) == 0:
        break

    generated_sentence.append(candidates[0])

```

generated_sentence

Output:

```

['the',
 'fulton',
 'county',
 'grand',
 'jury',
 'said',
 'friday',
 'an',
 'investigation',
 'of',
 'atlantas']

```

Step 7: Count Word Frequencies

```

from collections import Counter

unigram_freq = Counter()
bigram_freq = Counter()

```

```
for sent in tokenized_sentences:
    unigram_freq.update(sent)
    bigram_freq.update(list(ngrams(sent, 2)))
```

Step 8: Compute N-gram Probability (MLE)

```
word1 = 'the'
word2 = 'man'
```

```
mle_prob = bigram_freq[(word1, word2)] / unigram_freq[word1]
mle_prob Output:
0.00265824412971088
```

Step 9: Apply Laplace Smoothing

```
laplace_prob = (bigram_freq[(word1, word2)] + 1) / (unigram_freq[word1] + vocab_size)
laplace_prob Output:
0.0016086575021936238
```

Step 10: Predict Next Word

```
input_word = 'the'
max_prob = 0
next_word = None
```

```
for v in vocab:
    prob = (bigram_freq[(input_word, v)] + 1) / (unigram_freq[input_word] + vocab_size)
```

```
    if prob > max_prob:
        max_prob = prob
        next_word = v
```

```
next_word Output:
'first'
```

Step 11: Generate Output

```
ngram_output = [input_word, next_word]
ngram_output Output:
['the', 'first']
```

N-Gram Accuracy

```
correct = 0
total = 0
```

```
for sent in tokenized_sentences[:1000]:
    if len(sent) < 2:
        continue
```

```

prev_word = sent[-2]
actual_word = sent[-1]

max_prob = 0
predicted_word = None

for v in vocab:
    prob = (bigram_freq[(prev_word, v)] + 1) / (unigram_freq[prev_word] + vocab_size)

    if prob > max_prob:
        max_prob = prob
        predicted_word = v

if predicted_word == actual_word:
    correct += 1

total += 1

ngram_accuracy = correct / total
ngram_accuracy Output:
0.10040567951318459

```

HIDDEN MARKOV MODEL (HMM) Step

5: Define HMM Components

```

observations = vocab
transition = {}
emission = {} states =
set()

```

Step 6: POS Tagging (Hidden States) `nlTK.download('averaged_perceptron_tagger')`

```

tagged_sentences = [] for sent in
tokenized_sentences[:5000]:
tagged_sentences.append(nltk.pos_tag(sent))

```

Step 7: Estimate Transition & Emission Counts

```

from collections import Counter

transition_count = Counter() emission_count
= Counter()
state_count = Counter()

for sent in tagged_sentences:

```

```

prev_tag = None

for word, tag in sent:
    states.add(tag)    state_count[tag]
    += 1
    emission_count[(tag, word)] += 1

    if prev_tag is not None:
        transition_count[(prev_tag, tag)] += 1

    prev_tag = tag

```

Step 8: Calculate Transition & Emission Probabilities

```

(t1, t2) in transition_count:
    transition[(t1, t2)] = transition_count[(t1, t2)] / state_count[t1]

for (tag, word) in emission_count:    emission[(tag, word)] =
    emission_count[(tag, word)] / state_count[tag]

```

Step 9: Train HMM Model

```

model_states
= list(states)

```

Step 10: Predict Next Word

```

input_sentence
= ['the', 'man']
last_word = input_sentence[-1]

candidate_words = {}

for (tag, word) in emission:    if
    word == last_word:        for next_tag
    in model_states:          if (tag,
    next_tag) in transition:
        for w in observations:    if (next_tag, w) in emission:
            candidate_words[w] = transition[(tag, next_tag)] * emission[(next_tag, w)]

hmm_next_word = max(candidate_words, key=candidate_words.get)

```

Step 11: Generate Output

```

hmm_output = input_sentence + [hmm_next_word]

```

HMM Accuracy

```

correct = 0
total = 0

```

```
for sent in tagged_sentences[:500]:
    if len(sent) < 2:
        continue

    prev_word, prev_tag = sent[-2]
    actual_word, actual_tag = sent[-1]

    predictions = {}

    for w in vocab:
        if (prev_tag, w) in emission:
            predictions[w] = emission[(prev_tag, w)]

    if len(predictions) == 0:
        continue

    predicted_word = max(predictions, key=predictions.get)

    if predicted_word == actual_word:
        correct += 1

    total += 1

hmm_accuracy = correct / total
hmm_accuracy Output:
0.0
```

Result

- The **N-gram model** achieved approximately **10% accuracy**, as it predicts the next word based on frequent word sequences in the corpus.
- The **HMM model** showed **0% accuracy**, since it predicts words based on POS patterns, which rarely match the exact next word in the dataset.